

First-Order Optimization Methods

Gradient Descent, Momentum, Adaptive Methods, SGD, and Proximal Gradient

Yoontae Hwang

What this lecture is about

Most optimization problems in data science do not have a closed-form answer. Instead, we build a sequence of guesses:

$$x^0, x^1, x^2, \dots$$

that gradually improves the objective value.

Central question

How can we use local information, especially gradients, to move toward a good minimizer?

- ▶ **Gradient descent:** the baseline first-order method.
- ▶ **Momentum and adaptive methods:** faster and more practical variants.
- ▶ **SGD and proximal gradient:** tools for large-scale and nonsmooth problems.

Learning goals

By the end, you should be able to:

- 1 Derive the gradient descent update from a first-order approximation.
- 2 Explain why step size controls both speed and stability.
- 3 Interpret smoothness, strong convexity, and condition number geometrically.
- 4 Compare GD, momentum, Nesterov, AdaGrad, RMSProp, Adam, SGD, and proximal GD.
- 5 Work through simple numerical examples by hand.
- 6 Recognize which optimizer idea is appropriate for common machine learning settings.

Guiding principle

The formulas matter, but the mental model matters more:

optimization = direction choice + step size choice + problem geometry.

Part I: Gradient descent

- 1 Stationarity and iterative algorithms
- 2 Descent directions and steepest descent
- 3 Step sizes and line search
- 4 Smoothness and convergence rates

Part II: Practical extensions

- 1 Momentum and Nesterov acceleration
- 2 Adaptive learning rates
- 3 Stochastic and minibatch gradients
- 4 Proximal gradient for nonsmooth terms

We will repeatedly return to simple examples such as quadratics, least squares, logistic regression, and Lasso.

The basic problem

We study unconstrained minimization:

$$\min_{x \in \mathbb{R}^n} f(x),$$

where f is usually differentiable.

Interpretation

- ▶ x is the decision variable or model parameter.
- ▶ $f(x)$ is the cost, loss, or objective value.
- ▶ A minimizer x^* satisfies $f(x^*) \leq f(x)$ for all x .

Example: least squares

$$f(\theta) = \frac{1}{2} \|A\theta - y\|_2^2.$$

Here θ is a model parameter and $f(\theta)$ measures prediction error.

Stationary points

For differentiable f , a point x is called **stationary** if

$$\nabla f(x) = 0.$$

Convex case

If f is convex and differentiable, then

$$x^* \text{ is a global minimizer} \iff \nabla f(x^*) = 0.$$

Nonconvex case

If f is not convex, $\nabla f(x) = 0$ may mean:

- ▶ local minimum,
- ▶ local maximum,
- ▶ saddle point.

So for nonconvex problems, first-order methods usually guarantee only approximate stationarity.

Worked example: stationary point of a convex quadratic

Consider

$$f(x) = \frac{1}{2}x^\top Qx - b^\top x, \quad Q \succ 0.$$

Gradient

$$\nabla f(x) = Qx - b.$$

Setting the gradient to zero gives

$$Qx^* - b = 0 \implies x^* = Q^{-1}b.$$

Why this is enough

Because $Q \succ 0$, the function is strongly convex. Therefore the stationary point is the unique global minimizer.

But in practice

For large Q , computing Q^{-1} is expensive or impossible. Iterative methods avoid direct inversion.

Worked example: stationarity does not always mean minimum

Consider the nonconvex function

$$f(x) = x^3.$$

Its derivative is

$$f'(x) = 3x^2.$$

Thus $x = 0$ is stationary.

But $x = 0$ is not a minimum

For $x < 0$, $x^3 < 0 = f(0)$. For $x > 0$, $x^3 > 0 = f(0)$. So $x = 0$ is a saddle-like flat point in one dimension.

Takeaway

For convex optimization, stationarity is a certificate of global optimality. For nonconvex optimization, stationarity is usually the best first-order target we can ask for.

The general iterative template

Almost every method in this lecture can be written as

$$x^{k+1} = x^k + \alpha^k d^k.$$

Two design choices

- ▶ d^k : the direction in which we move.
- ▶ $\alpha^k > 0$: the step size or learning rate.

Examples

- ▶ Gradient descent: $d^k = -\nabla f(x^k)$.
- ▶ Newton method: $d^k = -[\nabla^2 f(x^k)]^{-1} \nabla f(x^k)$.
- ▶ Momentum: d^k uses current and past gradients.

What information can an algorithm use?

At the current point x^k , an algorithm may query local information.

Function value

$$f(x^k)$$

Cheap, but gives little directional information.

Gradient

$$\nabla f(x^k)$$

First-order information. This is the workhorse of large-scale optimization.

Hessian

$$\nabla^2 f(x^k)$$

Curvature information. Powerful, but expensive in high dimensions.

First-order methods use function values and gradients, but not full Hessian inverses.

Why the negative gradient?

Near x , the first-order Taylor approximation is

$$f(x + \alpha d) \approx f(x) + \alpha \langle \nabla f(x), d \rangle.$$

If $\alpha > 0$, the linearized objective decreases when

$$\langle \nabla f(x), d \rangle < 0.$$

Steepest decrease under the Euclidean norm

Among directions with $\|d\|_2 \leq 1$, the best direction is

$$d^* = -\frac{\nabla f(x)}{\|\nabla f(x)\|_2}.$$

Mental picture

The gradient points uphill. The negative gradient points most directly downhill.

A first numerical gradient step

Let

$$f(x, y) = x^2 + 3y^2.$$

Then

$$\nabla f(x, y) = \begin{bmatrix} 2x \\ 6y \end{bmatrix}.$$

Start from $(x^0, y^0) = (2, 1)$ and use step size $\alpha = 0.1$.

Gradient descent update

$$\begin{aligned}\nabla f(2, 1) &= \begin{bmatrix} 4 \\ 6 \end{bmatrix}, \\ \begin{bmatrix} x^1 \\ y^1 \end{bmatrix} &= \begin{bmatrix} 2 \\ 1 \end{bmatrix} - 0.1 \begin{bmatrix} 4 \\ 6 \end{bmatrix} = \begin{bmatrix} 1.6 \\ 0.4 \end{bmatrix}.\end{aligned}$$

Objective value check

$$f(2, 1) = 7, \quad f(1.6, 0.4) = 2.56 + 0.48 = 3.04.$$

The step reduced the objective.

Descent directions

Definition

A direction d is a **descent direction** at x if there exists $\bar{\alpha} > 0$ such that

$$f(x + \alpha d) < f(x) \quad \text{for all } \alpha \in (0, \bar{\alpha}).$$

Simple sufficient condition

If

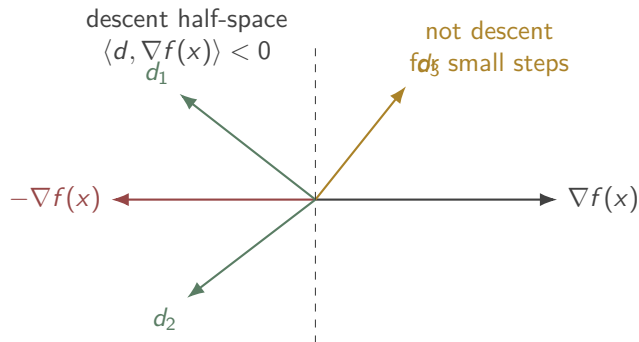
$$\langle d, \nabla f(x) \rangle < 0,$$

then d is a descent direction.

Meaning

The direction d forms an obtuse angle with the gradient. It does not have to be exactly the negative gradient.

Geometric picture of descent



Key idea

There are many valid descent directions. Different optimization methods choose different directions to exploit geometry, sparsity, or curvature.

Positive definite scaling

Suppose $B \succ 0$ and choose

$$d = -B\nabla f(x).$$

Then

$$\langle d, \nabla f(x) \rangle = -\nabla f(x)^\top B \nabla f(x) < 0$$

whenever $\nabla f(x) \neq 0$.

General scaled gradient update

$$x^{k+1} = x^k - \alpha^k B^k \nabla f(x^k).$$

Why this is useful

B^k can rescale the problem. If one coordinate is very steep and another is flat, a good B^k can move cautiously in the steep direction and boldly in the flat direction.

Many algorithms are scaled gradients

Method	Direction	Scaling idea
Gradient descent	$-\nabla f(x)$	$B = I$
Diagonal scaling	$-D\nabla f(x)$	coordinate-wise curvature
Newton method	$-\left[\nabla^2 f(x)\right]^{-1}\nabla f(x)$	local Hessian inverse
AdaGrad / Adam	adaptive scaled gradient	gradient-history scaling

Practical trade-off

More curvature information can improve convergence, but it also increases computation and memory.

Rule of thumb

For very high-dimensional machine learning, full Hessians are usually too expensive. First-order and diagonal-adaptive methods are preferred.

Steepest descent depends on the norm

The phrase “steepest direction” is incomplete unless we specify a norm:

$$d^* = \arg \min_{\|d\| \leq 1} \langle \nabla f(x), d \rangle .$$

ℓ_2 ball

Sphere-like geometry.

$$d^* \propto -\nabla f(x)$$

Gradient descent.

ℓ_1 ball

Diamond-like geometry.

d^* uses one coordinate

Coordinate descent.

ℓ_∞ ball

Cube-like geometry.

$$d^* = -\text{sign}(\nabla f(x))$$

Sign gradient descent.

Gradient descent from the ℓ_2 norm

With Euclidean geometry,

$$\arg \min_{\|d\|_2 \leq 1} \langle \nabla f(x), d \rangle = -\frac{\nabla f(x)}{\|\nabla f(x)\|_2}.$$

The usual update absorbs the magnitude into the step size:

$$x^{k+1} = x^k - \alpha^k \nabla f(x^k).$$

Example

If $\nabla f(x^k) = (4, 6)$ and $\alpha = 0.1$, then

$$x^{k+1} = x^k - (0.4, 0.6).$$

Weakness

The same step size is used in every coordinate. This can be slow when the problem has very different curvature in different directions.

Coordinate descent from the ℓ_1 norm

With the ℓ_1 unit ball, the steepest direction selects the coordinate with largest absolute partial derivative.

Greedy coordinate choice

$$i_k \in \arg \max_j \left| \frac{\partial f}{\partial x_j}(x^k) \right|.$$

Then update only coordinate i_k :

$$(x^{k+1})_{i_k} = (x^k)_{i_k} - \alpha^k \frac{\partial f}{\partial x_{i_k}}(x^k),$$

while all other coordinates stay fixed.

When coordinate descent is attractive

It can be efficient when each coordinate update is cheap, or when changing one variable at a time has a closed-form solution.

Worked example: one coordinate update

Let

$$f(x, y) = x^2 + 10y^2.$$

At $(x, y) = (3, 1)$,

$$\nabla f(3, 1) = \begin{bmatrix} 6 \\ 20 \end{bmatrix}.$$

Coordinate choice

Since $|20| > |6|$, coordinate descent updates y first.

With $\alpha = 0.05$,

$$y^+ = 1 - 0.05(20) = 0, \quad x^+ = 3.$$

Objective check

$$f(3, 1) = 9 + 10 = 19, \quad f(3, 0) = 9.$$

One coordinate step removed the steep-direction error.

Sign gradient descent from the ℓ_∞ norm

With the ℓ_∞ unit ball,

$$d^* = -\text{sign}(\nabla f(x)).$$

The update is

$$x^{k+1} = x^k - \alpha^k \text{sign}(\nabla f(x^k)).$$

What information is used?

Only the sign of each gradient component:

positive $\Rightarrow$ move down, negative $\Rightarrow$ move up.

Example

If $\nabla f(x) = (100, 0.02, -7)$, then

$$\text{sign}(\nabla f(x)) = (1, 1, -1).$$

The magnitudes are ignored.

Newton direction as curvature correction

Newton's method uses the local quadratic approximation

$$f(x + s) \approx f(x) + \langle \nabla f(x), s \rangle + \frac{1}{2} s^\top \nabla^2 f(x) s.$$

Minimizing the right-hand side gives

$$s = -[\nabla^2 f(x)]^{-1} \nabla f(x).$$

Newton update

$$x^{k+1} = x^k - [\nabla^2 f(x^k)]^{-1} \nabla f(x^k).$$

Cost

Computing and inverting the Hessian can be too expensive when n is large.

Worked example: Newton solves a quadratic in one step

Let

$$f(x) = \frac{1}{2}ax^2 - bx, \quad a > 0.$$

Then

$$f'(x) = ax - b, \quad f''(x) = a.$$

Newton's update is

$$x^+ = x - \frac{f'(x)}{f''(x)} = x - \frac{ax - b}{a} = \frac{b}{a}.$$

Interpretation

The minimizer is $x^* = b/a$. Newton reaches it in one step from any starting point because the quadratic model is exact.

Why not always use Newton?

For general high-dimensional problems, the Hessian may be expensive, singular, or indefinite.

Why step size matters

The direction tells us where to move. The step size tells us how far.

Too small

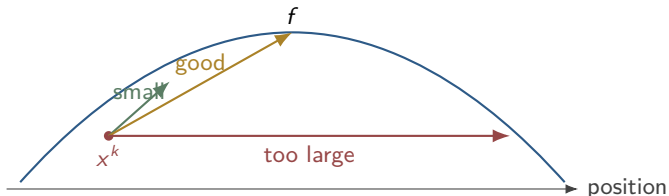
The method is stable but slow.

Well chosen

The method makes rapid progress without overshooting.

Too large

The method may oscillate or diverge.



One-dimensional quadratic analysis

Consider

$$f(x) = \frac{1}{2}ax^2, \quad a > 0.$$

Gradient descent gives

$$x^{k+1} = x^k - \alpha ax^k = (1 - a\alpha)x^k.$$

After k steps,

$$|x^k| = |1 - a\alpha|^k |x^0|.$$

Convergence condition

$$|1 - a\alpha| < 1 \quad \Longleftrightarrow \quad 0 < \alpha < \frac{2}{a}.$$

Best one-dimensional step size

If $\alpha = 1/a$, then $x^{k+1} = 0$ in one step.

Worked example: three step sizes

Let

$$f(x) = \frac{1}{2}(4)x^2 = 2x^2, \quad f'(x) = 4x.$$

Then convergence requires

$$0 < \alpha < \frac{2}{4} = 0.5.$$

Start from $x^0 = 1$.

Step size	multiplier $1 - 4\alpha$	behavior	first step
0.10	0.60	stable, slow	$x^1 = 0.60$
0.25	0.00	best here	$x^1 = 0$
0.60	-1.40	diverges	$x^1 = -1.40$

Lesson

A step that is safe for a flat function may be unsafe for a steep function.

Two-dimensional quadratic and zig-zagging

Consider

$$f(x, y) = \frac{1}{2}(ax^2 + by^2), \quad 0 < a < b.$$

Gradient descent gives

$$x^+ = (1 - a\alpha)x, \quad y^+ = (1 - b\alpha)y.$$

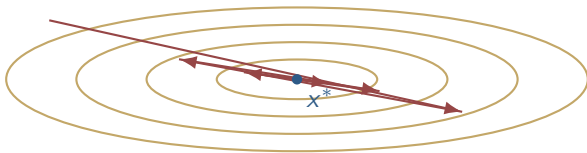
Different directions shrink at different speeds

- ▶ The y direction is steep because curvature is b .
- ▶ The x direction is flat because curvature is a .
- ▶ A large step may make the steep coordinate oscillate.

Zig-zag condition

If $1/b < \alpha < 2/b$, then $1 - b\alpha < 0$. The y coordinate changes sign at every step.

Visual intuition: elongated contours



Ill-conditioned problems have long narrow valleys.
Gradient descent often bounces across the valley.

Condition number intuition

Large condition number means the contours are stretched. This makes plain gradient descent slow.

Condition number

For a quadratic

$$f(x) = \frac{1}{2}x^\top Qx,$$

where $Q \succ 0$, the curvature in each eigen-direction is an eigenvalue of Q .

Definition

$$\kappa = \frac{\lambda_{\max}(Q)}{\lambda_{\min}(Q)}.$$

Interpretation

- ▶ $\kappa \approx 1$: contours are nearly circular, GD is fast.
- ▶ $\kappa \gg 1$: contours are elongated, GD zig-zags and slows down.

Mental model

Condition number measures how unfair the landscape is across directions.

Best constant step size on a 2D quadratic

For

$$f(x, y) = \frac{1}{2}(ax^2 + by^2), \quad a < b,$$

the contraction factors are

$$|1 - a\alpha|, \quad |1 - b\alpha|.$$

The worst factor is

$$\rho(\alpha) = \max\{|1 - a\alpha|, |1 - b\alpha|\}.$$

Optimal balancing

Choose α so the two worst errors are equal:

$$1 - a\alpha = -(1 - b\alpha) \implies \alpha^* = \frac{2}{a + b}.$$

Then

$$\rho^* = \frac{b - a}{b + a} = \frac{\kappa - 1}{\kappa + 1}.$$

Worked example: condition number and speed

Let $a = 1$ and $b = 100$, so

$$\kappa = \frac{100}{1} = 100.$$

The best constant step size is

$$\alpha^* = \frac{2}{1 + 100} = \frac{2}{101} \approx 0.0198.$$

The contraction factor is

$$\rho^* = \frac{100 - 1}{100 + 1} = \frac{99}{101} \approx 0.9802.$$

What this means

Each iteration reduces the distance by only about 2%. Even the best constant step size is slow when κ is large.

Motivation

This is why momentum and adaptive scaling become important.

Gradient descent as surrogate minimization

Define the quadratic surrogate

$$g_{\alpha}(z) = f(x^k) + \langle \nabla f(x^k), z - x^k \rangle + \frac{1}{2\alpha} \|z - x^k\|_2^2.$$

This is a linear approximation plus a penalty for moving too far.

Minimizer of the surrogate

Set the derivative with respect to z to zero:

$$\nabla f(x^k) + \frac{1}{\alpha}(z - x^k) = 0.$$

Therefore

$$z = x^k - \alpha \nabla f(x^k).$$

Takeaway

Gradient descent minimizes a simple local model of f at each step.

Worked example: surrogate on $f(x) = x^2$

Let $f(x) = x^2$ and current point $x^k = 2$. Then

$$f(2) = 4, \quad f'(2) = 4.$$

The surrogate is

$$g_\alpha(z) = 4 + 4(z - 2) + \frac{1}{2\alpha}(z - 2)^2.$$

If $\alpha = 0.25$

$$z = 2 - 0.25(4) = 1.$$

Objective check

$$f(2) = 4, \quad f(1) = 1.$$

The surrogate minimizer gives a real decrease.

If α is too large

The surrogate may no longer upper-bound the true objective, and the step may overshoot.

Three step-size strategies

Constant

$$\alpha^k = \alpha$$

Simple and common. Requires tuning or knowledge of smoothness.

Diminishing

$$\alpha^k \rightarrow 0, \quad \sum_k \alpha^k = \infty$$

Useful for noisy stochastic gradients.

Line search

Choose α^k during iteration by checking whether the objective decreases enough.

No single strategy is best for every problem.

Constant step size

The simplest choice is

$$\alpha^k = \alpha \quad \text{for all } k.$$

Advantages

- ▶ Easy to implement.
- ▶ No extra function evaluations.
- ▶ Common in large-scale training with a learning-rate schedule.

Disadvantages

- ▶ Too large can diverge.
- ▶ Too small can be painfully slow.
- ▶ The best value depends on curvature.

Theory preview

For L -smooth functions, $\alpha \leq 1/L$ gives a guaranteed decrease for gradient descent.

Diminishing step size

A diminishing sequence often satisfies

$$\alpha^k \rightarrow 0, \quad \sum_{k=1}^{\infty} \alpha^k = \infty.$$

Common examples:

$$\alpha^k = \frac{c}{k}, \quad \alpha^k = \frac{c}{\sqrt{k}}.$$

Why $\alpha^k \rightarrow 0$?

Eventually steps become cautious, which helps avoid overshooting and controls stochastic noise.

Why $\sum_k \alpha^k = \infty$?

The method never completely runs out of travel distance.

Backtracking line search

Backtracking chooses α adaptively by requiring sufficient decrease.

Armijo condition

Accept α if

$$f(x^k + \alpha d^k) \leq f(x^k) + c\alpha \langle \nabla f(x^k), d^k \rangle, \quad c \in (0, 1).$$

Algorithm

- 1 Start with $\alpha = 1$.
- 2 If Armijo fails, replace $\alpha \leftarrow \beta\alpha$ with $\beta \in (0, 1)$.
- 3 Repeat until the condition holds.

Cost

Line search uses extra function evaluations, so it can be costly in very large models.

Worked example: backtracking by hand

Let

$$f(x) = x^2, \quad x^k = 1, \quad d^k = -f'(1) = -2.$$

Use $c = 1/2$ and try $\alpha = 1$.

Check Armijo

Left side:

$$f(1 + 1(-2)) = f(-1) = 1.$$

Right side:

$$f(1) + \frac{1}{2}(1)(2)(-2) = 1 - 2 = -1.$$

So $1 \leq -1$ is false. Reject $\alpha = 1$.

Try $\alpha = 1/4$

$$f(1 - 1/2) = f(1/2) = 1/4,$$

while the Armijo right side is $1 + (1/2)(1/4)(-4) = 1/2$. Thus $1/4 \leq 1/2$: accept.

Learning-rate schedules in practice

Deep learning often uses a learning rate that changes over training.

Common schedules

- ▶ Step decay: drop by a factor after fixed epochs.
- ▶ Exponential decay: multiply by a constant each epoch.
- ▶ Cosine decay: smooth decrease from high to low.
- ▶ Warmup: start small, then increase.

Why warmup?

At initialization, gradients can be unstable. A small initial learning rate lets optimizer statistics stabilize before large updates.

Practical advice

If training diverges early, reduce the initial learning rate or add warmup. If training is stable but slow, consider increasing it or using a schedule.

L-smoothness

A differentiable function is **L-smooth** if

$$\|\nabla f(x) - \nabla f(y)\|_2 \leq L \|x - y\|_2 \quad \forall x, y.$$

Interpretation

The gradient cannot change too fast. Smoothness bounds curvature from above.

Quadratic example

For

$$f(x) = \frac{1}{2} x^\top Q x,$$

we have $\nabla^2 f(x) = Q$, so

$$L = \lambda_{\max}(Q).$$

Why it matters

Smoothness tells us how large a gradient step can be before the linear approximation becomes unreliable.

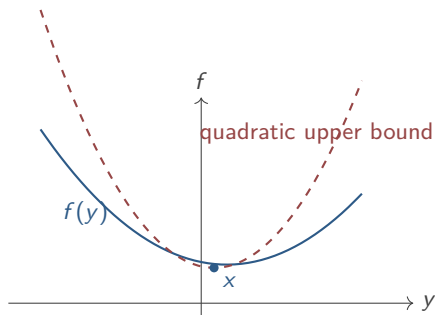
Quadratic upper bound from smoothness

If f is L -smooth, then for all x, y ,

$$f(y) \leq f(x) + \langle \nabla f(x), y - x \rangle + \frac{L}{2} \|y - x\|_2^2.$$

Meaning

A smooth function lies below a quadratic model centered at x .



μ -strong convexity

A differentiable function is μ -**strongly convex** if

$$f(y) \geq f(x) + \langle \nabla f(x), y - x \rangle + \frac{\mu}{2} \|y - x\|_2^2 \quad \forall x, y.$$

Interpretation

The function curves upward at least as fast as a quadratic bowl.

Quadratic example

For

$$f(x) = \frac{1}{2} x^\top Q x, \quad Q \succ 0,$$

we have

$$\mu = \lambda_{\min}(Q), \quad L = \lambda_{\max}(Q).$$

Consequence

A strongly convex function has a unique minimizer.

Worked example: what nonconvex GD can promise

Imagine training a neural network. The loss is usually nonconvex.

What the theory says

If the loss is smooth and the objective is bounded below, GD can drive

$$\|\nabla f(x^k)\|$$

toward a small value.

What it does not say

It does not prove that the point is the best possible neural network parameter among all parameters.

Practical interpretation

Small gradient means the training procedure has reached a point where local first-order changes no longer give an easy improvement.

Case: convex and smooth

If f is convex and L -smooth, with $\alpha = 1/L$, then

$$f(x^K) - f(x^*) \leq \frac{L \|x^0 - x^*\|_2^2}{2K}.$$

Rate

$$f(x^K) - f(x^*) = O\left(\frac{1}{K}\right).$$

Meaning

To make the objective gap at most ε , the number of iterations scales like

$$K = O\left(\frac{1}{\varepsilon}\right).$$

Convexity matters

Convexity lets us turn small-gradient progress into a global objective gap guarantee.

Why momentum?

Gradient descent can zig-zag in high-curvature directions. Momentum tries to smooth the trajectory by using previous steps.

Physical analogy

Imagine a ball rolling down a valley:

- ▶ repeated gradients in the same direction build speed;
- ▶ alternating gradients cancel out oscillations.

Optimization interpretation

Momentum uses past gradients to estimate a more stable direction of travel.

Caution

Momentum can accelerate convergence, but bad hyperparameters can cause overshooting or instability.

Heavy-ball momentum

The heavy-ball method updates by

$$x^{k+1} = x^k - \alpha \nabla f(x^k) + \beta(x^k - x^{k-1}), \quad 0 \leq \beta < 1.$$

Two terms

- ▶ $-\alpha \nabla f(x^k)$: current downhill step.
- ▶ $\beta(x^k - x^{k-1})$: keep moving in the previous direction.

Special case

If $\beta = 0$, heavy-ball reduces to ordinary gradient descent.

Nonmonotonicity

Momentum methods do not always decrease $f(x^k)$ at every single step.

Momentum as an exponential moving average

Define a velocity-like variable v^k . A common form is

$$\begin{aligned}v^{k+1} &= \beta v^k + \nabla f(x^k), \\x^{k+1} &= x^k - \alpha v^{k+1}.\end{aligned}$$

Unrolling the first recursion gives

$$v^{k+1} = \nabla f(x^k) + \beta \nabla f(x^{k-1}) + \beta^2 \nabla f(x^{k-2}) + \dots$$

Meaning

Recent gradients get large weights. Older gradients are exponentially discounted.

Typical values

In practice, $\beta = 0.9$ is a common starting point for momentum.

Worked example: momentum in one dimension

Let $f(x) = \frac{1}{2}x^2$, so $f'(x) = x$. Use $\alpha = 0.2$, $\beta = 0.8$, $x^0 = 1$, and $v^0 = 0$.

Step 1

$$v^1 = 0.8(0) + 1 = 1, \quad x^1 = 1 - 0.2(1) = 0.8.$$

Step 2

$$v^2 = 0.8(1) + 0.8 = 1.6, \quad x^2 = 0.8 - 0.2(1.6) = 0.48.$$

Observation

The second update is larger than plain GD would take at $x = 0.8$ because momentum accumulated past downhill direction.

Momentum on ill-conditioned quadratics

For a strongly convex quadratic with condition number κ , tuned heavy-ball momentum can achieve

$$O\left(\sqrt{\kappa} \log \frac{1}{\varepsilon}\right)$$

iterations, compared with

$$O\left(\kappa \log \frac{1}{\varepsilon}\right)$$

for plain gradient descent.

Why this is a big deal

If $\kappa = 10,000$, then

$$\kappa = 10,000, \quad \sqrt{\kappa} = 100.$$

This can be a dramatic improvement.

Heavy-ball optimal parameters for quadratics

For a quadratic with curvature bounds

$$ml \preceq A \preceq MI, \quad \kappa = \frac{M}{m},$$

the classical heavy-ball parameters are

$$\alpha = \left(\frac{2}{\sqrt{m} + \sqrt{M}} \right)^2, \quad \beta = \left(\frac{\sqrt{M} - \sqrt{m}}{\sqrt{M} + \sqrt{m}} \right)^2.$$

Resulting contraction

$$\rho = \frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1}.$$

Practical note

These formulas require knowing curvature bounds. In machine learning, α and β are usually tuned empirically.

Nesterov acceleration: look ahead first

Nesterov's accelerated gradient evaluates the gradient at a lookahead point.

Update

$$\begin{aligned}y^{k+1} &= x^k + \beta(x^k - x^{k-1}), \\x^{k+1} &= y^{k+1} - \alpha \nabla f(y^{k+1}).\end{aligned}$$

Difference from heavy-ball

- ▶ Heavy-ball: gradient at current point, then add momentum.
- ▶ Nesterov: move to lookahead point, then compute gradient there.

Why it matters

This small change gives strong convergence guarantees for general smooth convex functions.

Heavy-ball vs. Nesterov

Heavy-ball

$$x^{k+1} = x^k - \alpha \nabla f(x^k) + \beta(x^k - x^{k-1}).$$

Uses the gradient at x^k .

Nesterov

$$y^{k+1} = x^k + \beta(x^k - x^{k-1}),$$

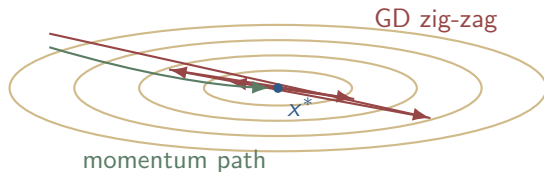
$$x^{k+1} = y^{k+1} - \alpha \nabla f(y^{k+1}).$$

Uses the gradient at the lookahead point.

Takeaway

Both accelerate GD, but Nesterov acceleration is the standard theoretically optimal method for smooth convex optimization.

Visual intuition: momentum smooths zig-zags



Message

Momentum cancels repeated back-and-forth movement and reinforces consistent forward movement.

Why adaptive methods?

Momentum still uses one global learning rate. But different coordinates can have very different scales.

Problem

A single α may be:

- ▶ too large for steep coordinates;
- ▶ too small for flat coordinates.

Adaptive idea

Scale each coordinate by information from past gradients:

$$x_j^{k+1} = x_j^k - \text{coordinate-specific step size} \times g_j^k.$$

Common methods

AdaGrad, RMSProp, Adam, and AdamW.

AdaGrad

AdaGrad accumulates squared gradients coordinate by coordinate:

$$G_j^k = \sum_{t=1}^k (g_j^t)^2.$$

Then

$$x_j^{k+1} = x_j^k - \alpha \frac{g_j^k}{\sqrt{G_j^k + \varepsilon}}.$$

Interpretation

If coordinate j has repeatedly large gradients, its effective step size becomes smaller.

Good for sparse features

Rarely updated features can get relatively larger steps. This is useful in text and recommender systems.

Worked example: AdaGrad scaling

Suppose at some iteration

$$g^k = (10, 1), \quad G^k = (10000, 25), \quad \alpha = 0.1, \quad \varepsilon \approx 0.$$

AdaGrad update is approximately

$$\Delta x_j = -0.1 \frac{g_j^k}{\sqrt{G_j^k}}.$$

Coordinate updates

$$\Delta x_1 = -0.1 \frac{10}{100} = -0.01, \quad \Delta x_2 = -0.1 \frac{1}{5} = -0.02.$$

Observation

Even though g_1 is larger, coordinate 1 gets a smaller update because it has accumulated much larger past gradients.

AdaGrad limitation

Because

$$G_j^k = G_j^{k-1} + (g_j^k)^2,$$

G_j^k only increases.

Potential issue

The effective step size

$$\frac{\alpha}{\sqrt{G_j^k + \epsilon}}$$

can become extremely small late in training.

Motivation for RMSProp

Instead of remembering all past squared gradients forever, use a moving average that forgets old gradients.

RMSProp

RMSProp replaces the cumulative sum by an exponential moving average:

$$v_j^k = \beta v_j^{k-1} + (1 - \beta)(g_j^k)^2, \quad \beta \approx 0.9.$$

Then

$$x_j^{k+1} = x_j^k - \alpha \frac{g_j^k}{\sqrt{v_j^k + \epsilon}}.$$

Interpretation

RMSProp estimates the recent scale of gradients in each coordinate.

Why it helps

If the landscape changes during training, RMSProp can adapt because old gradients are gradually forgotten.

Adam: momentum plus RMSProp

Adam keeps two moving averages:

$$\begin{aligned}m^k &= \beta_1 m^{k-1} + (1 - \beta_1) g^k, \\v^k &= \beta_2 v^{k-1} + (1 - \beta_2) (g^k)^2.\end{aligned}$$

Then update coordinatewise:

$$x^{k+1} = x^k - \alpha \frac{\hat{m}^k}{\sqrt{\hat{v}^k} + \varepsilon}.$$

Roles

- ▶ m^k : smoothed gradient direction, like momentum.
- ▶ v^k : smoothed squared gradient scale, like RMSProp.

Typical defaults

$$\beta_1 = 0.9, \quad \beta_2 = 0.999, \quad \varepsilon = 10^{-8}.$$

Why Adam uses bias correction

At the beginning, the moving averages start at zero:

$$m^0 = 0, \quad v^0 = 0.$$

Therefore early estimates are biased toward zero.

Bias-corrected estimates

$$\hat{m}^k = \frac{m^k}{1 - \beta_1^k}, \quad \hat{v}^k = \frac{v^k}{1 - \beta_2^k}.$$

Intuition

Bias correction prevents Adam from taking artificially small early steps just because the moving-average buffers started at zero.

Worked example: one Adam step

Use scalar gradient $g^1 = 4$, $m^0 = v^0 = 0$, $\beta_1 = 0.9$, $\beta_2 = 0.999$.

Raw moving averages

$$m^1 = 0.9(0) + 0.1(4) = 0.4, v^1 = 0.999(0) + 0.001(4^2) = 0.016.$$

Bias correction

$$\hat{m}^1 = \frac{0.4}{1 - 0.9} = 4, \quad \hat{v}^1 = \frac{0.016}{1 - 0.999} = 16.$$

Thus the normalized direction is

$$\frac{\hat{m}^1}{\sqrt{\hat{v}^1}} = \frac{4}{4} = 1.$$

AdamW: decoupled weight decay

For standard SGD, L2 regularization and weight decay are equivalent. For Adam, they are not equivalent because Adam rescales gradients coordinatewise.

AdamW idea

Do not put $\lambda\theta$ into the gradient. Apply decay directly:

$$\theta^{k+1} = (1 - \alpha\lambda)\theta^k - \alpha \frac{\hat{m}^k}{\sqrt{\hat{v}^k} + \varepsilon}.$$

Why this is cleaner

Weight decay shrinks all parameters uniformly instead of being distorted by Adam's adaptive denominator.

Practical note

AdamW is widely used for training modern neural networks, especially transformer-based models.

Choosing among GD, momentum, and Adam

Situation	Reasonable method	Why
Small clean convex problem	GD or Newton variant	simple, stable
Ill-conditioned smooth problem	momentum / Nesterov	reduces zig-zagging
Sparse features	AdaGrad	rare features get larger steps
Deep neural networks	AdamW or SGD+momentum	practical and scalable
Noisy finite-sum loss	minibatch SGD variants	cheap gradient estimates
Nonsmooth regularizer	proximal gradient	handles h exactly

No free lunch

Optimizer choice depends on scale, noise, curvature, regularization, and compute budget.

Finite-sum objectives

Many machine learning losses have the form

$$f(\theta) = \frac{1}{m} \sum_{i=1}^m f_i(\theta),$$

where f_i is the loss on the i -th data point.

Examples

- ▶ Linear regression: $f_i(\theta) = \frac{1}{2}(\mathbf{x}_i^\top \theta - y_i)^2$.
- ▶ Logistic regression: $f_i(\theta)$ is cross-entropy loss.
- ▶ Neural networks: $f_i(\theta) = \ell(\text{model}_\theta(x_i), y_i)$.

Problem

When m is huge, computing

$$\nabla f(\theta) = \frac{1}{m} \sum_{i=1}^m \nabla f_i(\theta)$$

every iteration is expensive.

Stochastic gradient descent

SGD samples one data point and uses its gradient:

$$i^k \sim \text{Uniform}\{1, \dots, m\}, \quad \theta^{k+1} = \theta^k - \alpha^k \nabla f_{i^k}(\theta^k).$$

Unbiasedness

$$\mathbb{E}[\nabla f_{i^k}(\theta)] = \frac{1}{m} \sum_{i=1}^m \nabla f_i(\theta) = \nabla f(\theta).$$

Trade-off

Each step is cheap but noisy. Full GD is expensive but deterministic.

Worked example: unbiased gradient estimate

Suppose a dataset has three per-example gradients at a fixed θ :

$$g_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad g_2 = \begin{bmatrix} 3 \\ 2 \end{bmatrix}, \quad g_3 = \begin{bmatrix} 2 \\ 4 \end{bmatrix}.$$

The full gradient is

$$\nabla f(\theta) = \frac{1}{3}(g_1 + g_2 + g_3) = \begin{bmatrix} 2 \\ 2 \end{bmatrix}.$$

SGD estimate

Sample one of g_1, g_2, g_3 uniformly.

$$\mathbb{E}[g_i] = \frac{1}{3}(g_1 + g_2 + g_3) = \begin{bmatrix} 2 \\ 2 \end{bmatrix}.$$

But one sample can be far from the average

That is the source of stochastic noise.

Noise in SGD

We can write a stochastic gradient as

$$g^k = \nabla f(x^k) + \xi^k, \quad \mathbb{E}[\xi^k] = 0.$$

The update becomes

$$x^{k+1} = x^k - \alpha^k \nabla f(x^k) - \alpha^k \xi^k.$$

Cumulative noise

Over many steps, noise accumulates roughly like

$$\sum_{k=1}^K \alpha^k \xi^k.$$

Why diminishing steps help

If α^k decreases, later noise is multiplied by smaller coefficients.

SGD convergence intuition

For convex stochastic optimization, a common result is

$$\mathbb{E}[f(\bar{x}^K)] - f(x^*) = O\left(\frac{1}{\sqrt{K}}\right),$$

with a step size on the order of $1/\sqrt{k}$.

Compared with full GD

- ▶ Full GD on smooth convex problems: $O(1/K)$ iterations.
- ▶ SGD with noisy gradients: $O(1/\sqrt{K})$ in expectation.

Why SGD is still useful

One SGD step can be much cheaper than one full gradient step. Wall-clock time may be better even if the iteration rate is slower.

Minibatch SGD

Instead of one data point, sample a batch B_k of size b :

$$g^k = \frac{1}{b} \sum_{i \in B_k} \nabla f_i(x^k).$$

Then

$$x^{k+1} = x^k - \alpha^k g^k.$$

Special cases

- ▶ $b = 1$: SGD.
- ▶ $b = m$: full gradient descent.

Variance reduction

Averaging b independent gradients reduces variance roughly by a factor of b .

Worked example: minibatch average

Using the previous gradients

$$g_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad g_2 = \begin{bmatrix} 3 \\ 2 \end{bmatrix}, \quad g_3 = \begin{bmatrix} 2 \\ 4 \end{bmatrix},$$

suppose the minibatch is $B = \{1, 3\}$.

Minibatch gradient

$$g_B = \frac{1}{2}(g_1 + g_3) = \frac{1}{2} \begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 1.5 \\ 2 \end{bmatrix}.$$

Comparison

The full gradient is $(2, 2)$. The minibatch estimate is not exact, but it is closer than many single-sample estimates.

Epochs, iterations, and batch size

In machine learning, people often distinguish iterations and epochs.

Definitions

- ▶ **Iteration:** one parameter update.
- ▶ **Epoch:** one pass through the whole dataset.

If the dataset has m examples and the batch size is b , then approximately

$$\text{iterations per epoch} = \frac{m}{b}.$$

Example

If $m = 50,000$ and $b = 100$, then one epoch is about

$$50,000/100 = 500$$

updates.

Practical SGD checklist

Symptoms and possible fixes

- ▶ Loss explodes: lower learning rate, add warmup, check gradients.
- ▶ Loss decreases but is noisy: increase batch size or use momentum.
- ▶ Training plateaus: decay learning rate or tune optimizer.
- ▶ Overfitting: use validation, weight decay, early stopping, data augmentation.

Important habit

Always plot training loss and validation loss. Optimization and generalization are related but not the same.

Why proximal methods?

Gradient descent assumes the objective is differentiable. But many important objectives are nonsmooth or constrained.

Lasso

$$\min_{\theta} \frac{1}{2} \|X\theta - y\|_2^2 + \lambda \|\theta\|_1.$$

The ℓ_1 term is not differentiable when a coordinate is zero.

Constrained least squares

$$\min_{\theta} \frac{1}{2} \|X\theta - y\|_2^2 \quad \text{s.t.} \quad \theta \in \mathcal{C}.$$

The constraint can be represented by an indicator function.

Goal

Extend gradient descent while handling nonsmooth structure exactly.

Composite objectives

A composite objective has the form

$$f(x) = g(x) + h(x),$$

where

- ▶ g is smooth and differentiable;
- ▶ h may be nonsmooth but has a simple proximal operator.

Lasso split

$$g(\theta) = \frac{1}{2} \|X\theta - y\|_2^2, \quad h(\theta) = \lambda \|\theta\|_1.$$

Constrained split

$$h(x) = I_{\mathcal{C}}(x) = \begin{cases} 0, & x \in \mathcal{C}, \\ +\infty, & x \notin \mathcal{C}. \end{cases}$$

Proximal mapping

The proximal mapping of h is

$$\text{prox}_{\alpha h}(x) = \arg \min_z \left\{ \frac{1}{2\alpha} \|z - x\|_2^2 + h(z) \right\}.$$

Interpretation

Find a point z that balances:

- ▶ staying close to x ;
- ▶ making $h(z)$ small.

Why this helps

If h has a closed-form prox, we can handle nonsmooth regularization without differentiating it.

Proximal gradient descent

For

$$\min_x g(x) + h(x),$$

proximal gradient performs two steps.

Algorithm

$$\begin{aligned} z^{k+1} &= x^k - \alpha^k \nabla g(x^k), \\ x^{k+1} &= \text{prox}_{\alpha^k h}(z^{k+1}). \end{aligned}$$

Meaning

- ▶ First, take a usual gradient step on the smooth part g .
- ▶ Then, apply the proximal operator to handle the nonsmooth part h .

Soft thresholding for the ℓ_1 penalty

For

$$h(x) = \lambda \|x\|_1,$$

the proximal operator is soft thresholding:

$$[\mathcal{S}_{\alpha\lambda}(x)]_j = \text{sign}(x_j) \max\{|x_j| - \alpha\lambda, 0\}.$$

Coordinate form

$$[\mathcal{S}_t(x)]_j = \begin{cases} x_j - t, & x_j > t, \\ 0, & |x_j| \leq t, \\ x_j + t, & x_j < -t. \end{cases}$$

Why it creates sparsity

Small coordinates are set exactly to zero.

Worked example: soft thresholding

Let

$$x = (3, 0.5, -2, -0.1), \quad t = 1.$$

Soft thresholding gives

$$\mathcal{S}_1(x) = (3 - 1, 0, -2 + 1, 0) = (2, 0, -1, 0).$$

What happened?

- ▶ Large positive entries shrink down.
- ▶ Large negative entries shrink toward zero.
- ▶ Entries with magnitude at most 1 become exactly zero.

Lasso via proximal gradient

For Lasso,

$$\min_{\theta} \frac{1}{2} \|X\theta - y\|_2^2 + \lambda \|\theta\|_1,$$

we set

$$g(\theta) = \frac{1}{2} \|X\theta - y\|_2^2, \quad h(\theta) = \lambda \|\theta\|_1.$$

Gradient of the smooth part

$$\nabla g(\theta) = X^\top (X\theta - y).$$

Proximal gradient update

$$\theta^{k+1} = \mathcal{S}_{\alpha\lambda} \left(\theta^k - \alpha X^\top (X\theta^k - y) \right).$$

Projection as a proximal operator

For the indicator function

$$h(x) = I_C(x),$$

the proximal operator becomes

$$\text{prox}_{\alpha h}(x) = \arg \min_{z \in \mathcal{C}} \|z - x\|_2^2 = P_C(x).$$

Projected gradient descent

$$x^{k+1} = P_C \left(x^k - \alpha \nabla g(x^k) \right).$$

Interpretation

First move as if unconstrained. Then project back to the feasible set.

Worked example: projection onto an interval

Let

$$\mathcal{C} = [0, 1].$$

The projection is

$$P_{[0,1]}(z) = \begin{cases} 0, & z < 0, \\ z, & 0 \leq z \leq 1, \\ 1, & z > 1. \end{cases}$$

Example

If a gradient step gives $z = 1.4$, then

$$P_{[0,1]}(1.4) = 1.$$

If it gives $z = -0.2$, then

$$P_{[0,1]}(-0.2) = 0.$$

Meaning

Projection enforces the constraint after every step.

Proximal gradient convergence

If g is convex and L -smooth, and h is convex, then proximal gradient with $\alpha = 1/L$ satisfies

$$f(x^k) - f(x^*) = O\left(\frac{1}{k}\right).$$

Remarkable point

Even though $f = g + h$ may be nonsmooth, the rate matches gradient descent on a smooth convex objective.

Why this works

The nonsmooth part is handled exactly by the proximal step rather than approximated by an ordinary gradient.

Method comparison

Method	Key idea	Best for
GD	negative gradient	baseline smooth optimization
Coordinate descent	update one coordinate	cheap coordinate steps
Newton	Hessian inverse scaling	small/medium well-behaved problems
Heavy-ball	past-step momentum	ill-conditioned quadratics
Nesterov	lookahead gradient	smooth convex acceleration
AdaGrad	cumulative adaptive scaling	sparse features
RMSProp	recent adaptive scaling	changing gradient scales
AdamW	momentum + adaptivity + decay	deep learning
SGD	noisy cheap gradients	massive datasets
Proximal GD	gradient + prox	nonsmooth regularizers

Common mistakes and how to diagnose them

Mistake 1: learning rate too large

Symptoms: loss becomes NaN, explodes, or oscillates wildly. Fix: reduce α , add warmup, clip gradients if needed.

Mistake 2: learning rate too small

Symptoms: loss decreases very slowly. Fix: increase α , use momentum, use adaptive methods, rescale features.

Mistake 3: ignoring conditioning

Symptoms: zig-zagging and slow convergence. Fix: normalize data, precondition, use momentum or adaptive learning rates.

Mini-check questions

- 1 For $f(x) = \frac{1}{2}ax^2$, what range of α makes GD converge?
- 2 Why does a large condition number slow down gradient descent?
- 3 What is the difference between heavy-ball and Nesterov momentum?
- 4 Why can AdaGrad's effective learning rate become too small?
- 5 Why is SGD noisy but computationally attractive?
- 6 What does the proximal operator of $\lambda \|x\|_1$ do?

Suggested study method

For each algorithm, memorize not only the update formula but also the failure mode it was designed to fix.

Answers to the mini-check

- 1 $0 < \alpha < 2/a$.
- 2 Different directions have different curvature, so a single step size cannot be optimal for all directions.
- 3 Heavy-ball computes the gradient at x^k ; Nesterov computes it at a lookahead point.
- 4 AdaGrad keeps accumulating squared gradients, so denominators grow monotonically.
- 5 SGD avoids full-gradient cost by sampling, but the sampled gradient has variance.
- 6 Soft thresholding: shrink coordinates and set small ones exactly to zero.

One-sentence summary

First-order optimization is the art of using gradients efficiently despite curvature, noise, scale, and nonsmoothness.